💙

AI 에이전트

AI 에이전트 구축에 대한 이해

1. AI 에이전트란 무엇인가

AI 에이전트는 대규모 언어모델(LLM)이 도구를 사용하면서 스스로 판단하고 행동을 반복해 목표를 달성하는 시스템이다.

OpenAI는 에이전트를 사용자를 대신해 작업을 독립적으로 수행하는 시스템으로 정의한다(OpenAI, A practical guide to building agents). 질문 하나에 답 하나를 내놓는 일반 챗봇과 달리, 에이전트는 목표가 달성될 때까지 추론과 행동을 되풀이한다.

Anthropic은 이런 시스템을 통틀어 "에이전틱 시스템"이라 부르고, 그 안에서 두 가지를 구분한다(Anthropic, Building effective agents).

- **워크플로**: 개발자가 미리 짜 둔 코드 경로를 따라 LLM과 도구를 조율하는 방식
- **에이전트**: LLM이 처리 순서와 도구 사용을 스스로 결정하는 방식

이 구분이 중요한 이유는 구축 방식과 비용, 위험이 달라지기 때문이다. 이 보고서는 에이전트의 구성 요소, 동작 원리, 구축 절차, 프레임워크, 활용 사례, 한계 순으로 정리한다.

2. 핵심 구성 요소

에이전트는 크게 모델, 도구, 지시문의 세 요소로 이루어진다. OpenAI의 가이드도 가장 기본적인 에이전트를 이 세 요소로 설명한다(OpenAI). Anthropic은 여기에 검색과 메모리를 더한 "증강된 LLM"을 모든 에이전틱 시스템의 기본 단위로 본다(Anthropic).

구성 요소	역할	설계 시 고려점
모델(LLM)	상황을 이해하고 다음 행동을 결정하는 두뇌	처음에는 성능이 높은 모델로 기준선을 잡고, 이후 비용과 지연을 줄이려 작은 모델로 바꾼다
지시문 (Instructions)	에이전트의 역할, 규칙, 예외 처리 방법을 적은 글	단계를 분명히 쓰고, 정보가 빠졌을 때의 분기까지 포함한다

구성 요소	역할	설계 시 고려점
도구(Tools)	검색, 데이터베이스 조회, API 호출 등 외부 세계에 작용하는 함수	정의를 표준화하고 설명을 명확히 쓴다. 도구가 늘면 에이전트를 나누는 것을 검토한다
검색·메모리	외부 지식을 가져오고 대화 간 상태를 유지하는 장치	필요한 정보만 컨텍스트에 넣어 비용과 혼선을 줄인다

도구 설계는 프롬프트만큼 중요하다. Anthropic은 코딩 에이전트를 만들 때 프롬프트보다 도구를 다듬는 데 더 많은 시간을 썼다고 소개한다(Anthropic).

도구를 에이전트에 연결하는 방식으로는 모델 컨텍스트 프로토콜(MCP)이 널리 쓰인다. MCP는 Anthropic이 2024년 11월 공개한 개방형 표준으로, AI 애플리케이션이 외부 도구와 데이터에 연결하는 방법을 하나로 맞춘다(IBM, What is MCP). 서버는 도구(실행 기능), 리소스(읽을 데이터), 프롬프트(재사용 지침)를 제공한다. MCP는 에이전트 프레임워크가 아니라 도구 연결 계층이며, 표준을 쓴다고 연동이 안전해지는 것은 아니다.

3. 동작 원리와 아키텍처

에이전트는 추론, 행동, 관찰을 목표를 달성할 때까지 되풀이하는 루프로 동작한다. 이 구조의 바탕은 ReAct 연구로, 모델이 추론 과정과 도구 사용을 번갈아 생성하면 추론은 행동 계획을 세우고 고치는 데 쓰이고, 행동은 외부에서 정보를 가져오는 데 쓰인다고 설명한다(ReAct 논문).

[embed: node/dca0385c-8353]

모델이 다음 행동을 정하면 도구가 실행되고, 그 결과를 관찰한 모델이 다시 판단하며, 목표를 달성하면 최종 답변으로 끝난다.

에이전트를 여러 개 조합하는 방식에는 두 가지가 있다. OpenAI의 가이드는 중앙의 관리자 에이전트가 전문 에이전트에게 일을 맡기는 관리자 패턴과, 에이전트끼리 제어권을 직접 넘기는 분산 패턴을 소개한다(Maginative). OpenAI와 Anthropic 모두 단일 구조로 시작하고 필요할 때만 복잡도를 높이라고 권한다.

Anthropic은 에이전트에 앞서 쓸 수 있는 워크플로를 다섯 가지 패턴으로 정리한다.

패턴	구조	적합한 경우
프롬프트 체이닝	한 호출의 출력을 다음 호출의 입력으로 넘기는 순차 구조	고정된 하위 단계로 깔끔히 나뉘는 작업
라우팅	입력을 분류해 전문 처리 경로로 보냄	유형별로 따로 처리하는 편이 나은 작업
병렬화	하위 작업을 동시에 처리하거나 같은 작업을 여러 번 돌려 비교	속도나 결과의 신뢰도를 높일 때

패턴	구조	적합한 경우
오케스트레이터-워커	중앙 LLM이 작업을 쪼개 워커에게 맡기고 결과를 종합	필요한 하위 작업을 미리 알 수 없을 때
평가자-최적화	한 LLM이 답을 만들고 다른 LLM이 평가해 개선을 반복	명확한 평가 기준이 있는 작업

4. 구축 절차

구축의 원칙은 가장 단순한 방식에서 시작해, 필요할 때만 자율성을 늘리는 것이다. Anthropic은 가장 성공적인 사례들이 복잡한 프레임워크보다 단순하고 조합 가능한 패턴을 썼다고 정리한다(Anthropic). OpenAI도 작게 시작해 일찍 검증하고 점차 확장하는 반복 접근을 권한다(babl.ai 요약).

1. **문제 정의와 적합성 판단**: 규칙 기반 자동화로 풀기 어려운 복잡하고 모호한 판단이 있는 업무인지 먼저 따진다. 그렇지 않다면 결정론적 해결책으로 충분하다(OpenAI).
2. **가장 단순한 구조 선택**: 단일 LLM 호출에 검색과 예시를 붙이는 것으로 충분한지 확인하고, 부족하면 워크플로, 그다음에 에이전트로 올라간다. 에이전트는 지연과 비용이 늘어나는 대가를 치른다(Anthropic).
3. **모델 선택**: 가장 성능 좋은 모델로 정확도 목표를 달성한 뒤, 작은 모델로 바꿔 비용과 지연을 줄인다.
4. **도구 정의**: 각 도구의 이름, 입력, 출력, 사용 시점을 명확히 적는다. 도구 설명은 사람이 읽는 API 문서처럼 쓴다.
5. **지시문 작성**: 역할, 처리 순서, 예외 상황의 분기를 구체적으로 쓴다.
6. **오케스트레이션 설계**: 단일 에이전트로 시작하고, 도구가 많아지거나 역할이 갈릴 때만 여러 에이전트로 나눈다.
7. **가드레일과 사람 개입 설계**: 입력·출력 검사를 겹겹이 두고, 환불이나 금융 정보 처리 같은 고위험 행동은 사람에게 넘기도록 만든다.
8. **평가·모니터링·개선**: 실제 업무 사례로 성공률, 비용, 지연을 측정하고, 실패한 사례를 지시문과 도구 설계에 반영해 반복한다.

5. 주요 프레임워크와 플랫폼

프레임워크는 에이전트의 반복 구조, 상태 관리, 도구 연결을 대신 처리해 주지만, Anthropic은 먼저 LLM API를 직접 써 보고, 프레임워크를 쓰더라도 내부 코드를 이해하라고 권한다(Anthropic). 아래 표는 2026년 비교 자료들을 바탕으로 한 개요이며, 이 분야는 변화가 빨라 선택 전에 공식 문서로 최신 상태를 확인해야 한다.

프레임워크	방식	강점	유의점
LangGraph	그래프로 단계와 분기를 명시	복잡한 상태를 유지하는 에이전트, 높은 제어력	규모가 커지면 디버깅이 어렵다는 평가가 있다
CrewAI	역할을 가진 에이전트 팀	프로토타입을 빠르게 만들 수 있다	기업 규모에서는 비용이 커질 수 있다
OpenAI Agents SDK	최소한의 기본 요소	가벼운 파이썬 에이전트에 적합	복잡한 상태 관리에는 LangGraph가 더 적합하다는 평가가 있다
Google ADK	그래프형 워크플로, 다중 언어(Python, Go, TypeScript)	구글 클라우드와 Gemini 연동	구글 생태계 밖의 모델 지원은 아직 초기 단계라는 평가가 있다
AutoGen → Microsoft Agent Framework	다중 에이전트 대화	다중 에이전트 협업 연구에 활용	AutoGen은 더 이상 활발히 개발되지 않아 신규 구축은 후속 프레임워크를 권한다는 보고가 있다

프레임워크 외에 **MCP**는 도구 연결의 표준으로, 최근 비교 자료들은 에이전트 간 통신 규약(A2A)과 함께 프레임워크 선택 기준으로 다룬다(RaftLabs). 한 자료는 프레임워크를 고르기 전에 어떤 방식(워크플로 중심인지 자율 에이전트인지)으로 만들지부터 정해야 재작성을 피할 수 있다고 지적한다(Eastgate).

6. 활용 사례와 경영정보 관점의 시사점

에이전트는 판단이 복잡하고 규칙이 자주 바뀌거나 비정형 데이터를 다루는 업무에서 효과가 크다. OpenAI의 가이드는 이런 특징을 가진 업무로 보험금 청구 처리를 예로 들고, 환불 처리·리서치·글쓰기 에이전트를 도구 또는 하위 에이전트의 예로 든다(OpenAI).

- **고객 지원**: 문의를 분류하고 주문·결제 시스템을 조회해 환불 같은 처리를 수행한다. 큰 금액의 환불은 사람에게 넘기도록 설계한다(babl.ai).
- **소프트웨어 개발**: 코드 저장소를 읽고 수정하며 테스트를 돌리는 코딩 에이전트가 대표적이다. 실제 GitHub 이슈 해결 능력은 SWE-bench로 측정한다(mer.vin).
- **리서치와 문서 작성**: 여러 출처를 검색하고 종합해 보고서 초안을 만든다.
- **업무 자동화**: 사내 시스템의 데이터를 조회하고 갱신하는 반복 업무를 처리한다.

시장 전망도 낙관과 경고가 함께 나온다. Gartner는 2028년까지 일상적인 업무 의사결정의 15%가 에이전틱 AI로 자율 처리되고, 기업용 소프트웨어의 33%가 에이전틱 AI를 포함할 것으로 예측한다(Gartner). 경영정보 관점에서 핵심은 기술 자체보다 업무 프로세스를 어떻게 다시 설계하고 기존 시스템과 어떻게 통합하느냐이다.

7. 한계와 위험, 안전한 설계 원칙

에이전트는 글을 쓰는 데서 그치지 않고 실제 시스템에 행동을 가하기 때문에, 실수의 비용이 챗봇보다 크다. 주요 위험과 대응은 다음과 같다.

- **프롬프트 인젝션**: 웹페이지나 문서 속의 악의적 문장이 모델에게 명령으로 해석되는 문제다. OWASP의 LLM 애플리케이션 10대 위험에서 3년 연속 1위를 유지하고 있다(Infosecurity Magazine). 지시문과 신뢰할 수 없는 콘텐츠를 분리하고, 통제를 모델 바깥에서 강제해야 한다(NHI Mgmt Group).
- **과도한 권한(Excessive agency)**: 필요 이상의 기능, 권한, 자율성을 에이전트에 주는 문제로, OWASP 순위가 6위에서 3위로 올랐다. 권고 대응은 도구의 수, 각 도구의 기능, 부여한 권한을 최소화하는 것이다(Infosecurity Magazine).
- **환각과 오류 누적**: 모델이 틀린 내용을 만들어 내거나, 여러 단계를 거치며 오류가 쌓일 수 있다. ReAct 연구는 외부 도구로 사실을 확인하며 추론하면 이런 문제가 줄어든다고 보고했다(ReAct 논문).
- **비용과 지연**: 에이전트는 모델 호출이 반복되므로 성능이 좋아지는 대신 지연과 비용이 늘 수 있다(Anthropic).
- **도입 실패 위험**: Gartner는 비용 증가, 불분명한 사업 가치, 부족한 위험 통제 때문에 2027년 말까지 에이전틱 AI 프로젝트의 40% 넘게 중단될 것으로 예측한다. 또 기존 도구에 이름만 붙이는 "에이전트 워싱"이 많다고 지적한다(Gartner).

안전한 설계의 공통 원칙은 세 가지로 요약된다. 첫째, 최소 권한을 부여한다. 둘째, 규칙 기반 검사와 LLM 기반 분류기 등 보호 장치를 겹쳐서 쓴다(babl.ai). 셋째, 오류나 고위험 결정 상황에서는 사람에게 넘기도록 설계하고, 초기 배포 단계에서 특히 사람의 개입을 계획한다.

참고문헌

- Anthropic, Building effective agents (2024-12)
- OpenAI, A practical guide to building agents
- Yao, S. et al., ReAct: Synergizing Reasoning and Acting in Language Models (ICLR 2023, arXiv:2210.03629)
- Gartner, Gartner Predicts Over 40% of Agentic AI Projects Will Be Canceled by End of 2027 (2025-06-25)
- IBM, What is Model Context Protocol (MCP)?

- Infosecurity Magazine, Prompt Injection Remains Biggest LLM Risk, Despite Limited Incidents
- NHI Mgmt Group, OWASP Top 10 for LLMs shows identity gaps in agentic AI
- RaftLabs, AI agent framework comparison (2026-04)
- Eastgate Software, AI agent frameworks comparison
- babl.ai, OpenAI Releases Practical Guide to Building LLM Agents (OpenAI 가이드 요약)
- mer.vin, When Not to Build AI Agents (Anthropic 가이드 해설)